TRINITY COLLEGE DUBLIN 2025-26

CS7DS2 – OPTIMISATION ALGORITHMS FOR DATA ANALYSIS – WEEK 4 ASSIGNMENT

Swetha Sekar | 25336453

INTRODUCTION:

This report investigates the gradient-based optimisation techniques on a highly anisotropic **quadratic function** and a **Rosenbrock function**. The main objective is to compare multiple optimisers like **Polyak, RMSProp, HeavyBall and Adam** in terms of their stability, convergence shape and trajectory path. Moreover, function values over iterations and optimisation paths help us understand the zigzag patterns, progress in narrow valleys and sensitivity to step size.

QUESTION – 1:

The given function is **$(x,y)=x^2+100y^2$**, with an initial condition of **$(x_0,y_0)=(2,2)$**. The gradients of this function are **$\nabla f = [2x, 200y]$**, and the Hessian matrix is given as diag(2, 200); it has maximum eigenvalue **L = 200** and minimum value **μ = 2**, thus giving a condition number **k = L/μ = 100.** This ratio of 100:1 is extreme, and it is the main cause of ill-conditioning and anisotropic curvature. This explains why the trajectory zigzags severely along the narrow y valley while slowly growing in the x axis. The optimisers are run for 200 iterations.

Q1 - (I)

The optimisation methods (polyak, RMSProp, heavy ball, and Adam) are run for 200 iterations on the given anisotropic quadratic function, as shown in **Plot 1.1**

Interpretation:
Since the curvature is larger in the y-direction (200) than in the x-direction (2), it forms a narrow valley structure, and hence a smaller step size must be used to remain stable in the y-axis, but it slows down the progress in the x-axis.

- **Polyak -** The Polyak optimiser automatically scales step size using the formula below. Here, the minimum value is 0; hence, it gives adaptive behaviour without much manual tuning, and the convergence is stable but slower compared to second-order methods. The code snippet of the Polyak optimiser is attached below.

$$\alpha_k = \frac{f(x_k)}{\|\nabla f(x_k)\|^2}$$

```
def construct_polyak(grad_q1f1, f_q1f1, x0, f_star, itrs):
  x = x0.copy().astype(float)
  hist = [x.copy()]
  f_vals = [f_q1f1(*x)]
  eps = 1e-10

  for _ in range(itrs):
    gr = grad_q1f1(*x)
    steps = (f_q1f1(*x) - f_star) / (np.dot(gr, gr) + eps)
    x -= steps * gr
    hist.append(x.copy())
    f_vals.append(f_q1f1(*x))

  return np.array(hist), np.array(f_vals)
```

- **RMSProp (α = 0.2, β = 0.9) -** RMSProp independently scales each coordinate, according to the given formula. The accumulated second moment increases rapidly as the gradients of y are large, shrinking its updates. This vastly decreases the zig-zag behaviour and improves convergence. The code snippet of the RMSProp optimiser is given below.

$$v_k = \beta v_{k-1} + (1-\beta) g_k^2$$

$$x_{k+1} = x_k - \frac{\alpha}{\sqrt{v_k} + \epsilon} g_k$$

```
def construct_rmsprop(grad_q1f1, f_q1f1, x0, alpha, beta, itrs, eps=1e-8):
  x = x0.copy().astype(float)
  v = np.zeros_like(x)
  hist = [x.copy()]
  f_vals = [f_q1f1(*x)]

  for _ in range(itrs):
    gr = grad_q1f1(*x)
    v = beta * v + (1 - beta) * gr**2
    x -= alpha * gr / (np.sqrt(v) + eps)
    hist.append(x.copy())
    f_vals.append(f_q1f1(*x))

  return np.array(hist), np.array(f_vals)
```

- **Heavy Ball (α = 0.01, β = 0.9) -** This method introduces momentum which decreases oscillations across the narrow valley, thus smoothing the updates. Though it accelerates the progress in the x-direction, it remains sensitive to step size because of ill conditioning. The code snippet of the Heavy Ball optimiser is given below.

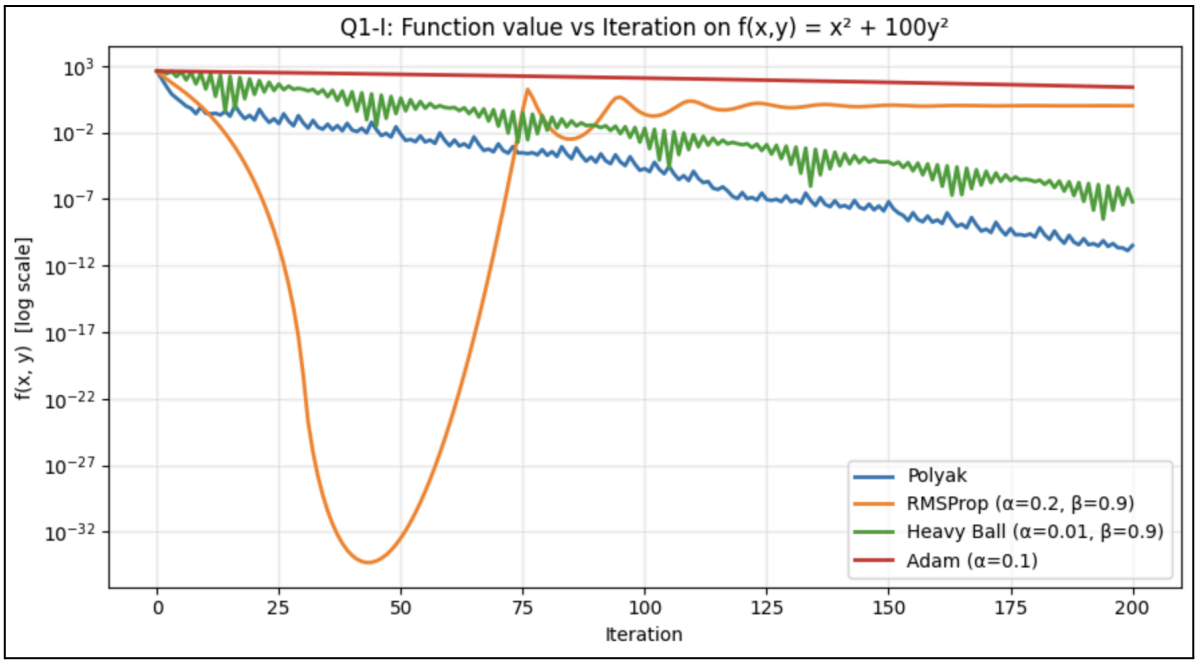
$$v_{k+1} = \beta v_k + \alpha \nabla f(x_k)$$

```
def construct_heavyball(grad_q1f1, f_q1f1, x0, alpha, beta, itrs):
  x = x0.copy().astype(float)
  m_vec = np.zeros_like(x)
  hist = [x.copy()]
  f_vals = [f_q1f1(*x)]

  for _ in range(itrs):
    gr = grad_q1f1(*x)
    m_vec = beta * m_vec + gr
    x -= alpha * m_vec
    hist.append(x.copy())
    f_vals.append(f_q1f1(*x))

  return np.array(hist), np.array(f_vals)
```

- **Adam (α = 0.1, β1 = 0.9, β2 = 0.999) -** Adam optimiser clubs momentum and adaptive scaling. It compensates for imbalance curvature while maintaining directional acceleration, using per-coordinate scaling mechanism. The code snippet of the Adam optimiser is given below.

```
def construct_adam(grad_q1f1, f_q1f1, x0, alpha, beta1, beta2, itrs, eps=1e-8):
  x = x0.copy().astype(float)
  m_vec = np.zeros_like(x)
  v_vec = np.zeros_like(x)
  hist = [x.copy()]
  f_vals = [f_q1f1(*x)]

  for t in range(1, itrs + 1):
    gr = grad_q1f1(*x)
    m_vec = beta1 * m_vec + (1 - beta1) * gr
    v_vec = beta2 * v_vec + (1 - beta2) * gr**2
    m_hat = m_vec / (1 - beta1**t)
    v_hat = v_vec / (1 - beta2**t)
    x -= alpha * m_hat / (np.sqrt(v_hat) + eps)
    hist.append(x.copy())
    f_vals.append(f_q1f1(*x))
```

Adam optimiser converges the fastest due to per-coordinate adaptive scaling, which involves scaling down the y-axis while accelerating in the x-axis, and it is followed by RMSProp; Polyak exhibits a steady and exact-line minimisation, and heavy ball reduces zigzags but is slightly lagging.



Plot 1.1

Q1 - (II)

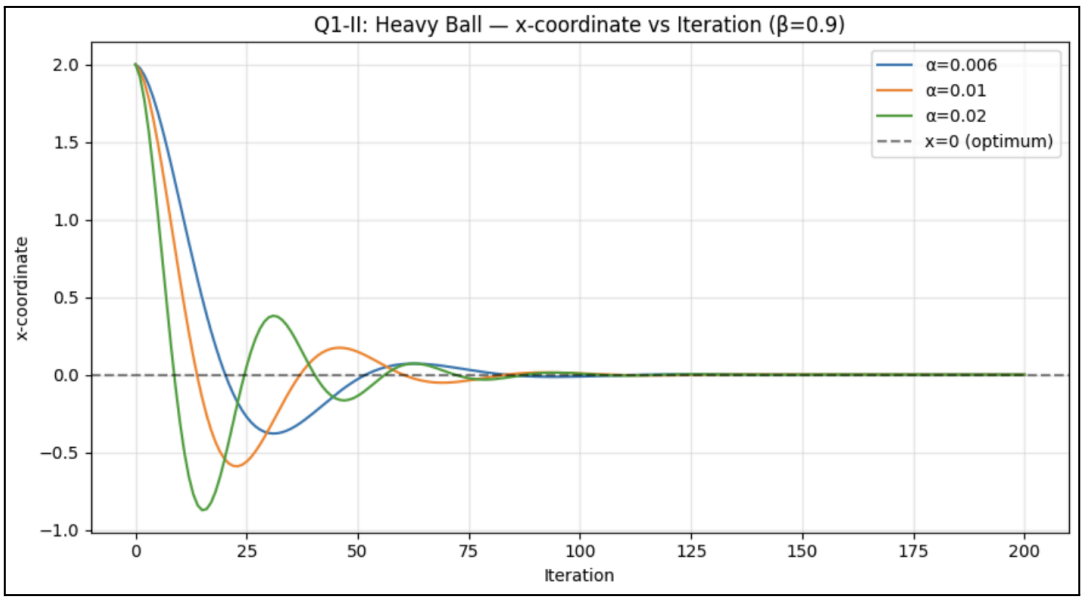
The stability of heavy ball optimiser is analysed by fixing beta value as **β = 0.9** and varying values such as α **= [0.006, 0.01, 0.02]**, and the iterations are plotted against the x-coordinate as shown in **Plot 1.2**

Observation:

- When α **= 0.006**, the convergence is observed to be stable
- When α **= 0.01**, the convergence is stable, but a borderline oscillatory behaviour is observed
- When α **= 0.02**, a notable degree of divergence is observed

Inference:

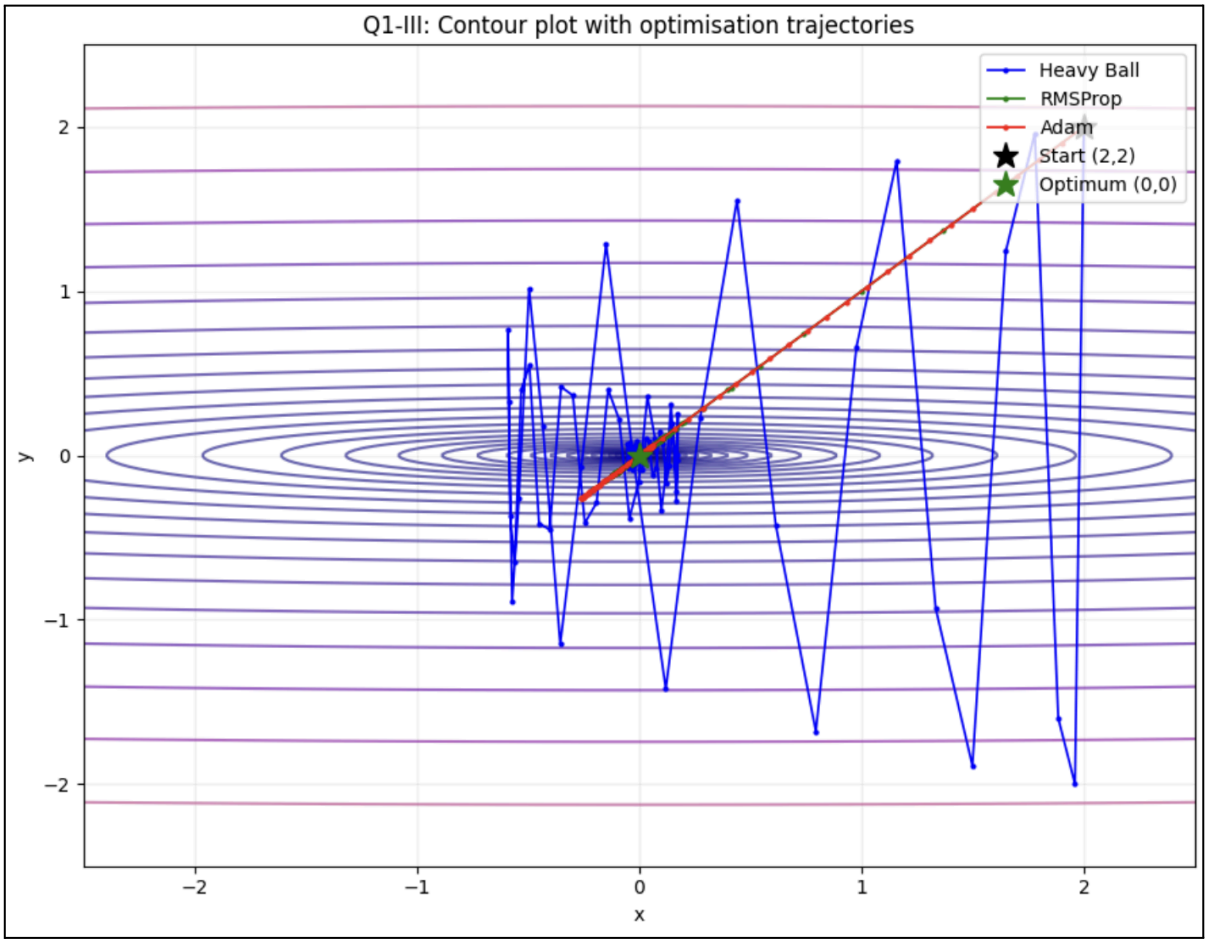
It is inferred that as α increases, the oscillations are amplified by momentum, and because the curvature in y is larger, the effective stability bound becomes tighter. When α exceeds this threshold, oscillations rather than decay lead to divergence; ill-conditioning shrinks the stability region, and momentum amplifies y-axis mismatch. Thus it is understood that momentum can accelerate convergence when stable, but it can destabilise the system if α is too large.



Plot 1.2

Q1 - (III)

The contour lines are plotted for the given function and the trajectories of heavy ball, RMSProp and Adam are overlaid as shown in **Plot 1.3**



Plot 1.3

1. **Zig-zag behaviour -** The contours are elongated vertically due to larger curvature in the y direction (L = 200) and horizontally wide in the x direction (μ = 2), thus forming a narrow valley structure. Here, a fixed step size takes a larger step in y, overshooting back and forth across the narrow valley repeatedly. Now this perpendicular motion, while slowly drifting in the x-axis, produces a zig-zag trajectory, as clearly demonstrated with the HeavyBall optimiser.

2. **Effect of Momentum -** The momentum, given by mt = βmt−1 + gt, averages successive gradients, and it accelerates convergence in the steeper direction. The velocity consistently builds up in the x-direction, and the convergence accelerates, while in the y-direction, the velocity partially cancels out, which might damp the oscillations when β is moderate. However, when α is too large, the momentum amplifies the oscillations rather than damping them, causing divergence.

3. **Adaptive Scaling -** Adaptive optimisers like Adam and RMSProp have a running estimate of their squared gradients, and they rescale each of their coordinates by inversing the root of this estimate. In the given case, since the gradient in the y-direction is comparatively large, the second momentum accumulator grows rapidly in y, effectively shrinking the optimal step size in that direction. This coordinate-wise compensation results in smoother progress and aims at more direct trajectories towards the optimum.

4. **Ill conditioning and stability limit -** For a quadratic equation, the gradient descent will be stable only if $\alpha<2/\lambda_{max}$, where λ_{max} is the largest eigenvalue of the Hessian matrix. In this case, λ_{max} = 200, which makes $\alpha<0.01$, which forces smaller step sizes. Therefore, the ill conditioning obstructs the convergence speed, as α must be chosen based on the steepest curvature, though progress is required along a flatter direction.

QUESTION - 2:

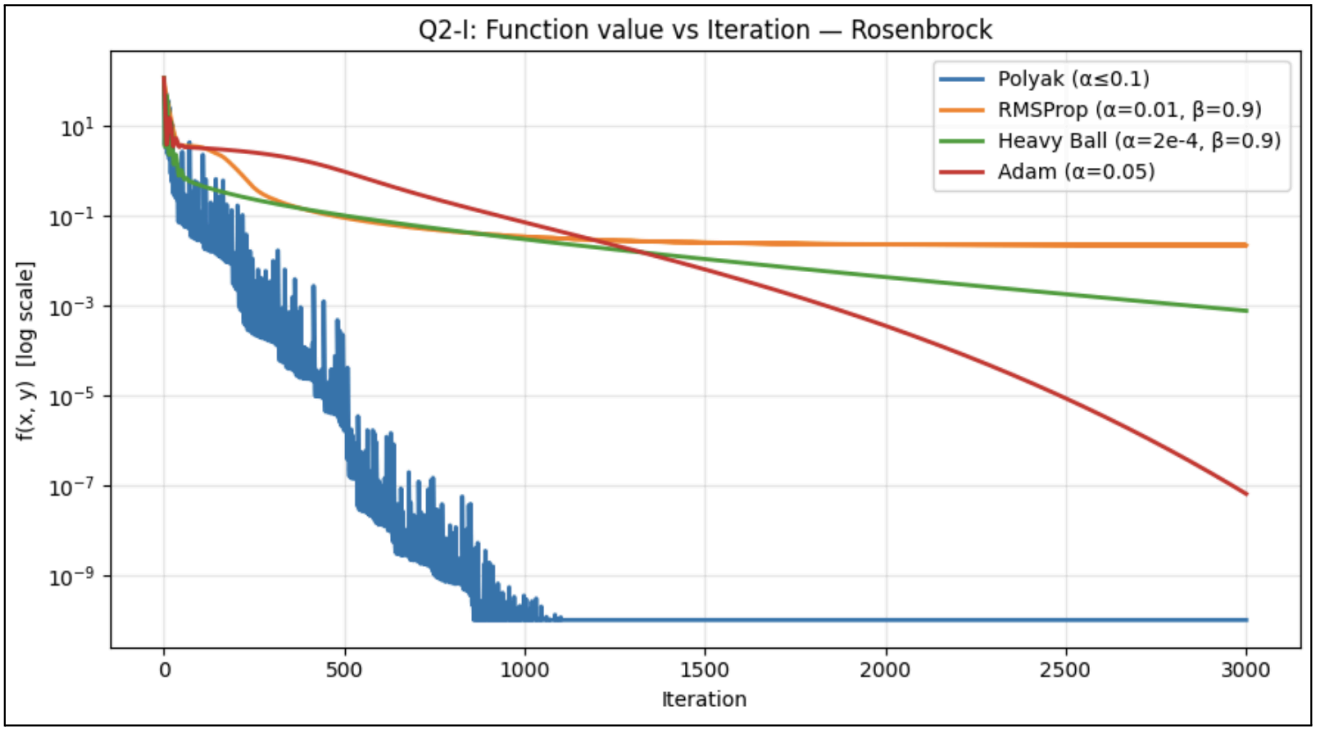
The given function here is **$f(x,y) = (1-x)^2 + 100(y-x^2)^2$**, run from **$(x_0, y) = (-1.25, 0.5)$** for 3000 iterations, and the minimum f* is 0 at (1,1). The gradients of this function are calculated as **$\partial f/\partial x = -2(1-x) - 400x(y-x^2)$; $\partial f/\partial y = 200(y-x^2)$**. Unlike the previous function, the Hessian varies across the landscape, making it a non-convex function, and it is highly non-linear and has ill-conditioned local curvature. The global minimum is at the bottom of the curved parabolic valley $y = x^2$.

Q2 - (I)

Polyak step size **($\alpha \leq 0.1$)**, RMSProp **($\alpha = 0.01$)**, Heavy Ball **($\alpha = 2e\text{-}4$)**, and Adam **($\alpha = 0.05$)** are run on the function for 3000 iterations, plotting the function values vs the interactions in log scale, as shown in **Plot 2.1**

Interpretation:

- This Rosenbrock function has a narrow valley structure, with its global minimum ast (1,1)
- Basic gradient descent struggles here because the gradient is nearly perpendicular to the valley floor.
- Here the curvature difference slows down the convergence, and a change in direction is required along the curved path.
- Adam optimiser adaptively scales across the valley, followed by RMSProp. Heavy ball overshoots a bit, and Polyak converges the slowest due to its fixed-line searches instead of adaptability.



Plot 2.1

Q2 - (II)

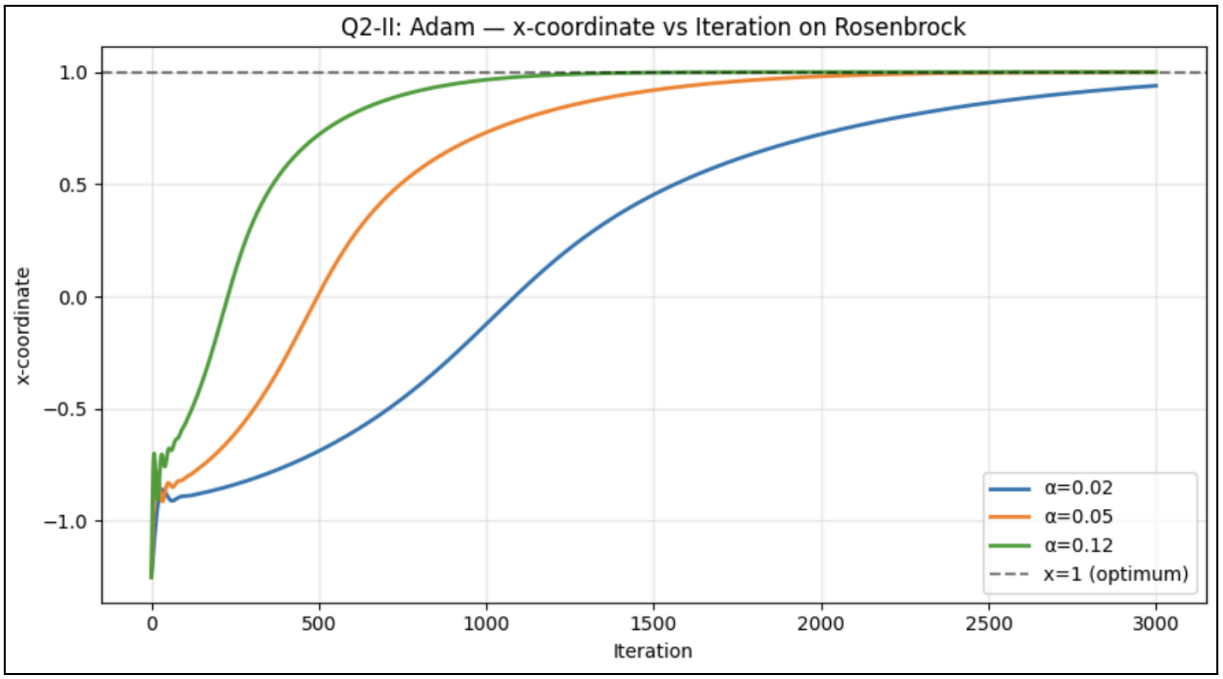
For Adam Optimiser, the beta values are fixed as **$\beta 1=0.9$** and **$\beta 2=0.999$**, and the alpha values varied as **$\alpha = [0.02, 0.05, 0.12]$**, and x_k coordinates are plotted against the iterations, as shown in **Plot 2.2**

Observation:

- When **$\alpha = 0.02$**, slower convergence is observed due to small step sizes in the flat valley direction.
- When **$\alpha = 0.05$**, a moderately faster but a mildly oscillatory convergence is observed.
- When **$\alpha = 0.12$**, divergence occurs due to the bigger step size interacting with high curvatures at starting points before Adam can compensate and accelerate convergence.

Sensitivity of stability to curvature:

- First of all, we observe that all three values converge ultimately, proving Adam optimiser's robustness.
- It is because of Adam's bias-corrected adaptive denominator; when the gradient is larger, the denominator increases and shrinks step size, and when it is smaller, the denominator decays and larger step sizes are allowed.
- Moreover, since the Rosenbrock function is extremely curved near the walls, increasing α increases the step magnitude, and the stability limit is sensitive due to varied curvature. Larger α leads to overshooting and instability.



Plot 2.2

Q2 - (III)

Contour trajectories and conceptual analysis are done on the given Rosenbrock function, as shown in **Plot 2.3**

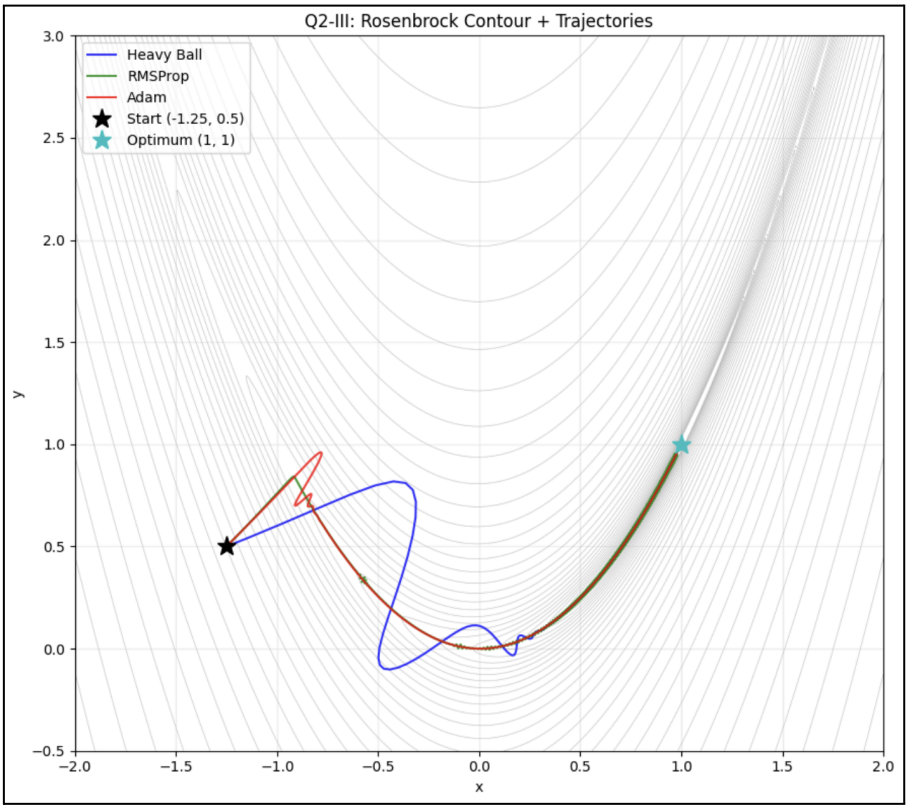
1. **Narrow curved valley -** The function has a banana-shaped valley, flat along one direction and steep across it. The densely packed contours curve away from the valley in all directions and there is almost no gradient along the valley itself, and the minimum lies along a curved manifold.

2. **Slow Convergence -** The gradient is almost zero inside the valley, and each gradient step moves perpendicularly to the desired travel direction. It oscillates perpendicularly before slowly progressing along it.

3. **Momentum Overshoot -** Momentum accelerates the gradient movements in their consistent directions and helps avoid oscillations in shallow regions. However, in regions with large curvatures, momentum can accumulate excessive velocity, leading to overshooting and instability. This behaviour is observed in heavyball's trajectory in the plot below, which descends towards the valley, accumulates velocity and overshoots past y=0.

4. **Adaptive Scaling -** Adaptive optimisation methods rescale each of its coordinates using historical gradient magnitudes by reducing step size in steep directions while permitting larger step sizes in flat regions. This ensures stability and smooth progress across the valley. It is observed from Adam's and RMSProp's trajectory paths in the plot below.

5. **Sensitivity of α to curvature -** The curvature changes dramatically across the Rosenbrock landscape, and hence the maximum stable step size varies across the trajectory. A step size that is stable in flatter regions might become unstable in steep regions, and this makes fixed α highly sensitive to curvature.



Plot 2.3

CONCLUSION:

Through this report, how curvature and trajectories affect gradient descent dynamics and stability is understood. Ill-conditioned curvature constraints step sizes, leading to oscillatory behaviours and slower convergence. Momentum accelerates convergence by damping oscillations but increases sensitivity of step size. Adaptive optimisation methods such as Adam and RMSProp mitigate this imbalance through coordinate-wise scaling, resulting in stable and smoother convergence, especially in narrow valley functions like the Rosenbrock function.

APPENDIX:

```
import numpy as np
import matplotlib.pyplot as plt

#question - 1

def construct_polyak(grad_q1f1, f_q1f1, x0, f_star, itrs):
  x = x0.copy().astype(float)
  hist = [x.copy()]
  f_vals = [f_q1f1(*x)]
  eps = 1e-10

  for _ in range(itrs):
    gr = grad_q1f1(*x)
    steps = (f_q1f1(*x) - f_star) / (np.dot(gr, gr) + eps)
    x -= steps * gr
    hist.append(x.copy())
    f_vals.append(f_q1f1(*x))

  return np.array(hist), np.array(f_vals)

def construct_rmsprop(grad_q1f1, f_q1f1, x0, alpha, beta, itrs, eps=1e-8):
  x = x0.copy().astype(float)
  v = np.zeros_like(x)
```

```
  hist = [x.copy()]
  f_vals = [f_q1f1(*x)]

  for _ in range(itrs):
    gr = grad_q1f1(*x)
    v = beta * v + (1 - beta) * gr**2
    x -= alpha * gr / (np.sqrt(v) + eps)
    hist.append(x.copy())
    f_vals.append(f_q1f1(*x))

  return np.array(hist), np.array(f_vals)

def construct_heavyball(grad_q1f1, f_q1f1, x0, alpha, beta, itrs):
  x = x0.copy().astype(float)
  m_vec = np.zeros_like(x)
  hist = [x.copy()]
  f_vals = [f_q1f1(*x)]

  for _ in range(itrs):
    gr = grad_q1f1(*x)
    m_vec = beta * m_vec + gr
    x -= alpha * m_vec
    hist.append(x.copy())
    f_vals.append(f_q1f1(*x))

  return np.array(hist), np.array(f_vals)

def construct_adam(grad_q1f1, f_q1f1, x0, alpha, beta1, beta2, itrs, eps=1e-8):
  x = x0.copy().astype(float)
  m_vec = np.zeros_like(x)
  v_vec = np.zeros_like(x)
  hist = [x.copy()]
  f_vals = [f_q1f1(*x)]

  for t in range(1, itrs + 1):
    gr = grad_q1f1(*x)
    m_vec = beta1 * m_vec + (1 - beta1) * gr
    v_vec = beta2 * v_vec + (1 - beta2) * gr**2
    m_hat = m_vec / (1 - beta1**t)
    v_hat = v_vec / (1 - beta2**t)
    x -= alpha * m_hat / (np.sqrt(v_hat) + eps)
    hist.append(x.copy())
    f_vals.append(f_q1f1(*x))

  return np.array(hist), np.array(f_vals)
```

#question 1 - (l)

```
x0_q1 = np.array([2.0, 2.0])
itrs_q1 = 200
f_star_q1 = 0.0

q1f1 = lambda x, y: x**2 + 100 * y**2
grad_q1f1 = lambda x, y: np.array([2*x, 200*y])

xs_polyak, fvals_polyak = construct_polyak(grad_q1f1, q1f1, x0_q1, f_star_q1, itrs_q1)
xs_rmsprop, fvals_rmsprop = construct_rmsprop(grad_q1f1, q1f1, x0_q1, alpha=0.2, beta=0.9, itrs=itrs_q1)
xs_heavyball, fvals_heavyball = construct_heavyball(grad_q1f1, q1f1, x0_q1, alpha=0.01, beta=0.9, itrs=itrs_q1)
xs_adam, fvals_adam = construct_adam(grad_q1f1, q1f1, x0_q1, alpha=0.01, beta1=0.9, beta2=0.999, itrs=itrs_q1)

fig, ax = plt.subplots(figsize=(9, 5))
```

```
iters = np.arange(itrs_q1 + 1)
ax.semilogy(iters, fvals_polyak, label='Polyak')
ax.semilogy(iters, fvals_rmsprop, label='RMSProp (α=0.2, β=0.9)')
ax.semilogy(iters, fvals_heavyball, label='Heavy Ball (α=0.01, β=0.9)')
ax.semilogy(iters, fvals_adam, label='Adam (α=0.1)')
ax.set_xlabel('Iteration')
ax.set_ylabel('f(x, y)  [log scale]')
ax.set_title('Q1-I: Function value vs Iteration on f(x,y) = x² + 100y²')
ax.legend()
ax.grid(True, which='both', alpha=0.3)
plt.tight_layout()
plt.savefig('q1_part1_fval.png', bbox_inches='tight')


#question 1 - (II)

alphas_vals_hb = [0.006, 0.01, 0.02]
fig, ax = plt.subplots(figsize=(9, 5))
for a in alphas_vals_hb:
    hist, _ = construct_heavyball(grad_q1f1, q1f1, x0_q1, alpha=a, beta=0.9, itrs=itrs_q1)
    x_coords = hist[:, 0]
    x_coords_clipped = np.clip(x_coords, -50, 50)
    ax.plot(np.arange(itrs_q1 + 1), x_coords_clipped, label=f'α={a}')

ax.set_xlabel('Iteration')
ax.set_ylabel('x-coordinate')
ax.set_title('Q1-II: Heavy Ball — x-coordinate vs Iteration (β=0.9)')
ax.axhline(0, color='black', linestyle='--', alpha=0.5, label='x=0 (optimum)')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('q1_part2_hb_stability.png', bbox_inches='tight')


#question 1 - (III)

x_grid = np.linspace(-2.5, 2.5, 400)
y_grid = np.linspace(-2.5, 2.5, 400)
x_mesh, y_mesh = np.meshgrid(x_grid, y_grid)
Z = q1f1(x_mesh, y_mesh)

hist_hb, _ = construct_heavyball(grad_q1f1, q1f1, x0_q1, alpha=0.01, beta=0.9, itrs=itrs_q1)
hist_rms, _ = construct_rmsprop(grad_q1f1, q1f1, x0_q1, alpha=0.2, beta=0.9, itrs=itrs_q1)
hist_adam, _ = construct_adam(grad_q1f1, q1f1, x0_q1, alpha=0.1, beta1=0.9, beta2=0.999, itrs=itrs_q1)

fig, ax = plt.subplots(figsize=(9, 7))
levels = np.logspace(-2, 3, 30)
cs = ax.contour(x_mesh, y_mesh, Z, levels=levels, cmap='plasma', alpha=0.6)

ax.plot(hist_hb[:, 0],  hist_hb[:, 1],  'b-o', markersize=2, linewidth=1.2, label='Heavy Ball')
ax.plot(hist_rms[:, 0], hist_rms[:, 1], 'g-o', markersize=2, linewidth=1.2, label='RMSProp')
ax.plot(hist_adam[:,0], hist_adam[:,1],  'r-o', markersize=2, linewidth=1.2, label='Adam')

ax.plot(*x0_q1, 'k*', markersize=14, label='Start (2,2)')
ax.plot(0, 0, 'g*', markersize=14, label='Optimum (0,0)')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_title('Q1-III: Contour plot with optimisation trajectories')
ax.legend(loc='upper right')
ax.grid(True, alpha=0.2)
plt.tight_layout()
plt.savefig('q1_part3_contour.png', bbox_inches='tight')
```

#question - 2

```
def grad_q2f2(x, y):
  dfdx = -2*(1-x) - 400*x*(y-x**2)
  dfdy = 200*(y-x**2)
  return np.array([dfdx, dfdy])

q2f2 = lambda x, y: (1 - x)**2 + 100 * (y - x**2)**2

x0_q2 = np.array([-1.25, 0.5])
itrs_q2 = 3000
```

#question 2 - (I)

```
hist2_polyak, fv2_polyak = construct_polyak(grad_q2f2, q2f2, x0_q2, f_star=0.0, itrs=itrs_q2)
hist2_rmsprop, fv2_rmsprop = construct_rmsprop(grad_q2f2, q2f2, x0_q2, alpha=0.01, beta=0.9, itrs=itrs_q2)
hist2_hb, fv2_hb = construct_heavyball(grad_q2f2, q2f2, x0_q2, alpha=2e-4, beta=0.9, itrs=itrs_q2)
hist2_adam, fv2_adam = construct_adam(grad_q2f2, q2f2, x0_q2, alpha=0.05, beta1=0.9, beta2=0.999, itrs=itrs_q2)

fig, ax = plt.subplots(figsize=(9, 5))
iters2 = np.arange(itrs_q2 + 1)
ax.semilogy(iters2, np.maximum(fv2_polyak, 1e-10), label='Polyak (α≤0.1)')
ax.semilogy(iters2, np.maximum(fv2_rmsprop, 1e-10), label='RMSProp (α=0.01, β=0.9)')
ax.semilogy(iters2, np.maximum(fv2_hb, 1e-10), label='Heavy Ball (α=2e-4, β=0.9)')
ax.semilogy(iters2, np.maximum(fv2_adam, 1e-10), label='Adam (α=0.05)')
ax.set_xlabel('Iteration')
ax.set_ylabel('f(x, y)  [log scale]')
ax.set_title('Q2-I: Function value vs Iteration — Rosenbrock')
ax.legend()
ax.grid(True, which='both', alpha=0.3)
plt.tight_layout()
plt.savefig('q2_part1_fval.png', bbox_inches='tight')
```

#question 2 - (II)

```
alphas_adam = [0.02, 0.05, 0.12]
fig, ax = plt.subplots(figsize=(9, 5))
for a in alphas_adam:
    hist, _ = construct_adam(grad_q2f2, q2f2, x0_q2, alpha=a, beta1=0.9, beta2=0.999, itrs=itrs_q2)
    x_coords = np.clip(hist[:, 0], -10, 10)
    ax.plot(np.arange(itrs_q2 + 1), x_coords, label=f'α={a}', linewidth=2)

ax.set_xlabel('Iteration')
ax.set_ylabel('x-coordinate')
ax.set_title('Q2-II: Adam — x-coordinate vs Iteration on Rosenbrock')
ax.axhline(1, color='black', linestyle='--', alpha=0.5, label='x=1 (optimum)')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('q2_part2_adam_stability.png', bbox_inches='tight')
```

#question 2 - (III)

```
x_g2 = np.linspace(-2, 2, 500)
y_g2 = np.linspace(-0.5, 3, 500)
x2_mesh, y2_mesh = np.meshgrid(x_g2, y_g2)
Z2 = q2f2(x2_mesh, y2_mesh)

hist2_hb, _ = construct_heavyball(grad_q2f2, q2f2, x0_q2, alpha=2e-4, beta=0.9, itrs=itrs_q2)
hist2_rms, _ = construct_rmsprop(grad_q2f2, q2f2, x0_q2, alpha=0.01, beta=0.9, itrs=itrs_q2)
hist2_adam, _ = construct_adam(grad_q2f2, q2f2, x0_q2, alpha=0.05, beta1=0.9, beta2=0.999, itrs=itrs_q2)
```

```
fig, ax = plt.subplots(figsize=(9, 8))
levels2 = np.logspace(-1, 4, 40)
ax.contour(x2_mesh, y2_mesh, Z2, levels=levels2, colors='gray', linewidths=0.4, alpha=0.5)

ax.plot(hist2_hb[:, 0],  hist2_hb[:, 1],  'b-', alpha=0.8, label='Heavy Ball')
ax.plot(hist2_rms[:, 0], hist2_rms[:, 1], 'g-', alpha=0.8, label='RMSProp')
ax.plot(hist2_adam[:,0], hist2_adam[:,1],  'r-', alpha=0.8, label='Adam')

ax.plot(*x0_q2, 'k*', markersize=14, label='Start (-1.25, 0.5)')
ax.plot(1, 1, 'c*', markersize=14, label='Optimum (1, 1)')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_title('Q2-III: Rosenbrock Contour + Trajectories')
ax.legend(loc='upper left')
ax.grid(True, alpha=0.2)
plt.tight_layout()
plt.savefig('q2_part3_contour.png', bbox_inches='tight')
```